Co-clustering of Ordinal Data: CUB Latent Block Model

Study notes for the project and the oral

Contents

1 The Project at a Glance **2**
- 1.1 Goal . . . 2

2 The CUB Distribution **2**
- 2.1 Definition . . . 2
- 2.2 Latent variable representation . . . 3
- 2.3 Estimation by EM (one CUB, counts c_r) . . . 3

3 The Latent Block Model **5**
- 3.1 Definition . . . 5
- 3.2 Likelihoods . . . 5
- 3.3 Counts per block . . . 6

4 The SEM-Gibbs Algorithm **6**
- 4.1 Principle . . . 6
- 4.2 SE step: the Gibbs conditionals . . . 6
- 4.3 M step . . . 7
- 4.4 Full algorithm . . . 7

5 Model Selection with ICL **9**
- 5.1 Number of free parameters . . . 9
- 5.2 ICL-BIC criterion . . . 9

6 Walk-through of the Code **9**
- 6.1 Key lines, explained . . . 10

7 Package Workflow **11**

8 Likely Oral Questions **11**

1 The Project at a Glance

1.1 Goal

We observe an $n \times d$ matrix $\underline{X} = (x_{ih})$ of ordinal data: each entry is a category $x_{ih} \in \{1, \dots, m\}$ (for instance a rating on a scale from 1 to m; rows are individuals, columns are items).

Co-clustering simultaneously

- groups the n rows into K row clusters,
- groups the d columns into L column clusters,

so that the matrix is summarised by $K \times L$ homogeneous blocks.

What we have to deliver. An R package whose main function takes the data, K, L and the number of random initialisations (20 by default) and returns the membership probabilities of rows and columns, the hard assignments, the ICL value and the estimated parameters.

The recipe.

Ingredient	Choice
Probabilistic model	Latent Block Model (LBM)
Distribution inside a block	CUB (ordinal)
Inference	SEM-Gibbs algorithm
Choice of (K, L)	ICL criterion
Robustness to local optima	several random initialisations

Why not simply EM? For the LBM the E step requires $P(z, w \mid x)$, which does not factorise (rows and columns are coupled). It is a sum over $K^n L^d$ configurations: intractable. SEM-Gibbs replaces this sum by simulation: we sample (z, w) with Gibbs sampling instead of computing expectations.

2 The CUB Distribution

2.1 Definition

For one ordinal variable $R \in \{1, \dots, m\}$, the CUB model (Combination of a Uniform and a shifted Binomial) is the mixture

$$\boxed{P(R = r) = \pi \binom{m-1}{r-1} \xi^{m-r} (1-\xi)^{r-1} + (1-\pi)\frac{1}{m}, \qquad r = 1, \dots, m.}$$

Interpretation. A respondent either

- answers with feeling (probability π): the answer follows a shifted binomial $1 + \mathcal{B}(m-1, 1-\xi)$, or
- answers with uncertainty (probability $1-\pi$): the answer is uniform over the m categories.

So π is the weight of the feeling component and ξ drives the location of the answers: the mean of the binomial part is $1 + (m-1)(1-\xi)$, hence a *large* ξ means *low* categories.

Check that it is a distribution. The binomial terms sum to one, $\sum_r \binom{m-1}{r-1}\xi^{m-r}(1-\xi)^{r-1} = 1$, and so do the uniform terms. Hence $\sum_r P(R=r) = \pi + (1-\pi) = 1$.

2.2 Latent variable representation

Introduce $V \sim \text{Bernoulli}(\pi)$ with

$$P(R = r \mid V = 1) = b(r;\xi) = \binom{m-1}{r-1}\xi^{m-r}(1-\xi)^{r-1}, \qquad P(R = r \mid V = 0) = \frac{1}{m}.$$

This turns CUB into a two-component mixture and allows an EM algorithm.

2.3 Estimation by EM (one CUB, counts c_r)

Let c_r be the number of observations equal to r, and $N = \sum_r c_r$.

E step. Posterior probability of "feeling" given $R = r$:

$$\boxed{\nu_r = \frac{\pi\, b(r;\xi)}{\pi\, b(r;\xi) + (1-\pi)/m}.}$$

M step. The expected complete log-likelihood is

$$\sum_r c_r \Big[\nu_r \big(\log\pi + \log b(r;\xi)\big) + (1-\nu_r)\big(\log(1-\pi) - \log m\big)\Big].$$

Maximising in π:

$$\boxed{\pi = \frac{\sum_r c_r \nu_r}{N}.}$$

In ξ, only $\sum_r c_r \nu_r \big[(m-r)\log\xi + (r-1)\log(1-\xi)\big]$ matters. Setting the derivative to zero and using $(m-r) + (r-1) = m-1$:

$$\boxed{\xi = \frac{\sum_r c_r \nu_r (m-r)}{(m-1)\sum_r c_r \nu_r}.}$$

Why counts? Only the m numbers c_r matter, not the individual data. This is the key efficiency trick of the code: for each block (k, ℓ) we tabulate the m counts and run this tiny EM on them.

Pitfalls.

- If $\pi \to 0$, ξ is not identifiable (the data are uniform); the code clamps π and ξ to $[10^{-6}, 1 - 10^{-6}]$.
- If a block is empty ($N = 0$), the parameters are kept unchanged.

3 The Latent Block Model

3.1 Definition

Latent variables:

- row labels $z_i \in \{1, \dots, K\}$, written $z_{ik} = \mathbb{1}\{z_i = k\}$, with $P(z_{ik} = 1) = \alpha_k$, $\sum_k \alpha_k = 1$;
- column labels $w_h \in \{1, \dots, L\}$, written $w_{h\ell} = \mathbb{1}\{w_h = \ell\}$, with $P(w_{h\ell} = 1) = \beta_\ell$, $\sum_\ell \beta_\ell = 1$;
- z and w are independent.

Given (z, w), the cells are independent and

$$x_{ih} \mid z_{ik} = 1,\, w_{h\ell} = 1 \;\sim\; \mathrm{CUB}(\pi_{k\ell}, \xi_{k\ell}).$$

The parameter is $\theta = (\alpha, \beta, \pi_{k\ell}, \xi_{k\ell})$.

3.2 Likelihoods

Complete-data likelihood.

$$p(\underline{X}, z, w; \theta) = \prod_{i,k} \alpha_k^{z_{ik}} \prod_{h,\ell} \beta_\ell^{w_{h\ell}} \prod_{i,h,k,\ell} p(x_{ih}; \pi_{k\ell}, \xi_{k\ell})^{z_{ik} w_{h\ell}},$$

so that

$$\boxed{\log p(\underline{X}, z, w; \theta) = \sum_{i,k} z_{ik} \log \alpha_k + \sum_{h,\ell} w_{h\ell} \log \beta_\ell + \sum_{i,h,k,\ell} z_{ik} w_{h\ell} \log p(x_{ih}; \pi_{k\ell}, \xi_{k\ell}).}$$

Observed-data likelihood.

$$p(\underline{X}; \theta) = \sum_{z \in \{1..K\}^n} \sum_{w \in \{1..L\}^d} p(\underline{X}, z, w; \theta),$$

a sum of $K^n L^d$ terms: impossible to compute directly.

Why EM fails. The E step needs $p(z, w \mid \underline{X})$ or at least the expectation of $z_{ik} w_{h\ell}$. Because z and w appear *together* in the product above, the posterior does not factorise over rows and columns. A variational approximation (VEM) is one remedy; the project uses simulation instead.

3.3 Counts per block

For a given pair of partitions (z, w) define

$$N_{k\ell r} = \sum_{i,h} z_{ik} w_{h\ell} \mathbb{1}\{x_{ih} = r\}, \qquad n_k = \sum_i z_{ik}, \qquad d_\ell = \sum_h w_{h\ell}.$$

Then the complete log-likelihood becomes

$$\boxed{\sum_k n_k \log \alpha_k + \sum_\ell d_\ell \log \beta_\ell + \sum_{k,\ell,r} N_{k\ell r} \log p(r; \pi_{k\ell}, \xi_{k\ell}).}$$

This is exactly what `complete_loglik()` computes in the code.

4 The SEM-Gibbs Algorithm

4.1 Principle

SEM = Stochastic EM. At every iteration

1. **SE step**: instead of computing expectations of the latent variables, simulate them from (an approximation of) their posterior, here with Gibbs sampling;
2. **M step**: maximise the complete-data likelihood at the simulated (z, w).

Because the latent variables are random, the parameter sequence is *not* converging to a point: it forms a Markov chain around a good region. We therefore discard a burn-in and average the parameters afterwards.

4.2 SE step: the Gibbs conditionals

Rows given columns. From the complete likelihood, keeping only the terms with z_i:

$$P(z_{ik} = 1 \mid \underline{X}, w; \theta) \;\propto\; \alpha_k \prod_{h=1}^{d} \prod_{\ell=1}^{L} p(x_{ih}; \pi_{k\ell}, \xi_{k\ell})^{w_{h\ell}}.$$

Hence $z_i \sim \mathcal{M}(1, \tilde{z}_{i1}, \ldots, \tilde{z}_{iK})$ with $\tilde{z}_{ik}$ the normalised right-hand side. Rows are independent given w.

Columns given rows. By symmetry,

$$P(w_{h\ell} = 1 \mid \underline{X}, z; \theta) \;\propto\; \beta_\ell \prod_{i=1}^{n} \prod_{k=1}^{K} p(x_{ih}; \pi_{k\ell}, \xi_{k\ell})^{z_{ik}}.$$

Log-space. Products of many small probabilities underflow, so the code works with scores

$$s_{ik} = \log \alpha_k + \sum_{\ell,r} \Big(\sum_{h} w_{h\ell} \mathbb{1}\{x_{ih} = r\} \Big) \log p(r; \pi_{k\ell}, \xi_{k\ell}),$$

and samples a category with probabilities $\propto e^{s_{ik}}$ (Gumbel-max trick: add independent Gumbel noise to s_{ik} and take the argmax; this gives exact categorical samples without normalising).

Matrix form (efficiency). Let $I_r = (\mathbb{1}\{x_{ih} = r\})_{ih}$ be the r-th indicator matrix, $W = (w_{h\ell})$ and $Z = (z_{ik})$. Then

$$S^{\text{row}} = \log \alpha^\top (\text{repeated}) + \sum_{r=1}^{m} (I_r W) \left(\log P_{\cdot\cdot r} \right)^\top, \qquad S^{\text{col}} = \log \beta^\top (\text{repeated}) + \sum_{r=1}^{m} (I_r^\top Z) \log P_{\cdot\cdot r},$$

where $\log P_{\cdot\cdot r}$ is the $K \times L$ matrix of $\log p(r; \pi_{k\ell}, \xi_{k\ell})$. No loop over cells is needed.

4.3 M step

Given the current (z, w):

$$\alpha_k = \frac{n_k}{n}, \qquad \beta_\ell = \frac{d_\ell}{d},$$

and $(\pi_{k\ell}, \xi_{k\ell})$ are obtained, block by block, by the inner CUB EM of the previous section on the counts $(N_{k\ell r})_r$ (warm start from the previous values).

4.4 Full algorithm

1. **Initialisation.** Row partition by k-means on the rows, column partition by k-means on the columns, random initial (π, ξ), then one M step.
2. **For** $t = 1, \ldots, T$ (default $T = 100$):
   (a) SE: `n_gibbs` sweeps (rows then columns).
   (b) M: update α, β, π, ξ.
   (c) If $t > T_{\text{burn}}$ (default 40), accumulate the parameters.
3. **Estimate** $\hat{\theta}$ = average of the accumulated parameters.
4. **Final chain.** Run Gibbs again with $\hat{\theta}$ fixed for T iterations; after burn-in, the frequency with which each row (column) was in each cluster gives the membership probabilities; the mode gives the final partition $(\hat{z}, \hat{w})$.

5. **ICL** at $(\hat{z}, \hat{w}, \hat{\theta})$.

6. Repeat from step 1 `n_init` times and keep the run with the highest ICL.

Remarks to be ready for the oral.

- Why several initialisations? SEM is random and the likelihood has many local optima.
- Why average parameters after burn-in? The chain does not converge to a point; the average is a stable estimate.
- Label switching: cluster numbers can be permuted between runs, so compare partitions with an invariant measure such as the Adjusted Rand Index.
- Empty clusters: the code moves a random member of the biggest cluster to any empty one, otherwise $\log \alpha_k = -\infty$ and the algorithm degenerates.

5 Model Selection with ICL

5.1 Number of free parameters

Parameters	Count
α (sums to one)	$K-1$
β (sums to one)	$L-1$
$(\pi_{k\ell}, \xi_{k\ell})$	$2KL$
Total ν	$(K-1)+(L-1)+2KL$

5.2 ICL-BIC criterion

The integrated completed likelihood penalises the complete-data log-likelihood. Each parameter is penalised by $\frac{1}{2}\log$(effective sample size): α is learned from n rows, β from d columns and each block parameter from the nd cells:

$$\boxed{\mathrm{ICL}(K,L) = \log p(\underline{X}, \hat{z}, \hat{w}; \hat{\theta}) - \frac{K-1}{2}\log n - \frac{L-1}{2}\log d - \frac{2KL}{2}\log(nd).}$$

The larger the ICL, the better the model.

Why ICL rather than BIC? BIC would need the observed likelihood (intractable here). ICL uses the *complete* likelihood, which we can compute at the estimated partitions, and it also favours well-separated clusters.

Remark on the paper. In the Jacques–Biernacki formula the mixing-proportion terms seem to be summed inside the (i,h) sum, which would count them nd times; the version above (each term once) is the standard one and the one implemented.

Best initialisation. For fixed (K,L) the penalty is the same for all initialisations, so choosing the best run by ICL is equivalent to choosing the largest complete log-likelihood.

6 Walk-through of the Code

All the code lives in `R/coclust_ordinal.R`. Reading order for the oral:

Function	Role
`coclust_ordinal`	Public entry point: checks inputs, builds the indicator matrices I_r, loops over `n_init`, keeps the best ICL.
`sem_gibbs_once`	One full SEM-Gibbs run: initialisation, SEM iterations, averaging, final chain, ICL.
`initial_partition`	k-means (or uniform random) starting partition.
`gibbs_sweep`	SE step: samples z given w, then w given z.
`draw_categorical`	Gumbel-max sampling from log-scores.
`fix_empty`	Repairs empty clusters.
`m_step`	Updates α, β and calls the CUB EM per block.
`fit_cub_counts`	Inner EM of a single CUB from m counts.
`log_cub_table`	$K \times L \times m$ array of $\log p(r; \pi_{k\ell}, \xi_{k\ell})$.
`block_counts`	$K \times L \times m$ array $N_{k\ell r}$ via $Z^\top I_r W$.
`complete_loglik`, `icl_value`	Complete log-likelihood and ICL.
`coclust_ordinal_select`	Grid search over (K, L) by ICL.
`simulate_cub_lbm`	Draws data from the model (tests, vignette).

6.1 Key lines, explained

CUB probability table.

```
out[, , r] <- log(pi * choose(m - 1, r - 1) * xi^(m - r) *
                    (1 - xi)^(r - 1) + (1 - pi) / m)
```

Direct transcription of the CUB formula, vectorised over all $K \times L$ blocks at once (π and ξ are $K \times L$ matrices).

Block counts.

```
N[, , r] <- crossprod(Z, Ilist[[r]] %*% W)
```

$Z^\top (I_r W)$ is the $K \times L$ matrix whose (k, ℓ) entry is $N_{k\ell r}$: the number of cells of block (k, ℓ) equal to r.

Row scores in the Gibbs sweep.

```
score <- score + (Ilist[[r]] %*% W) %*% t(matrix(logP[, , r], K, L))
```

$(I_r W)_{i\ell}$ counts how many cells of row i equal r within column cluster ℓ; multiplying by $\log p(r; \theta_{k\ell})$ and summing over ℓ and r gives s_{ik}.

Sampling.

```
g <- -log(-log(stats::runif(length(logscore))))
max.col(logscore + matrix(g, nrow(logscore)), ties.method = "first")
```

Gumbel-max trick: $\arg\max_k (s_{ik} + G_k)$ with G_k i.i.d. Gumbel has distribution $\propto e^{s_{ik}}$.

Inner EM.

```
nu <- pi * pb / (pi * pb + (1 - pi) / m)        # E step
s  <- sum(cnt * nu)
pi_new <- s / tot                                 # M step
xi_new <- sum(cnt * nu * (m - r)) / ((m - 1) * s)
```

Exactly the boxed formulas of the CUB section.

ICL.

```
loglik - (K - 1) / 2 * log(n) - (L - 1) / 2 * log(d) -
  K * L * log(n * d)
```

Note $\frac{2KL}{2}\log(nd) = KL\log(nd)$.

7 Package Workflow

1. `devtools::document()`: generates `man/*.Rd` and `NAMESPACE` from the roxygen comments (`#'`).
2. `devtools::test()`: runs the `testthat` tests.
3. `devtools::check()` / `R CMD check`: official package checks (documentation, examples, vignette).
4. `R CMD build .`: produces `coclustOrdinal_0.1.0.tar.gz`.
5. `install.packages("...tar.gz", repos = NULL, type = "source")` to install it (this is what the professor will do).

The package is pure R (no C++), so it installs on any Mac without a compiler.

8 Likely Oral Questions

1. What is a CUB distribution and what do π and ξ mean?
2. Why is EM impossible for the LBM, and what do we do instead?
3. Derive the conditional distribution of z_i given w.
4. Why do we average parameters after burn-in?
5. How are $\pi_{k\ell}$ and $\xi_{k\ell}$ updated?
6. How many parameters does the model have and why this ICL penalty?
7. What happens when a cluster becomes empty, or when $\pi \to 0$?
8. How do you choose among the random initialisations?

9. How are membership probabilities obtained from a stochastic algorithm?

10. What is label switching and how do you compare two partitions?

11. Why vectorise with indicator matrices instead of looping over cells?